

Contents

1	Introduction	3
2	Objectives	3
3	System Architecture	4
3.1	Frontend Module	4
3.2	Backend Module	4
3.3	Architecture Flow	5
4	Technologies Used	5
5	REST API Design	5
5.1	API Endpoints	5
5.2	REST Communication Flow	6
6	Encryption Methodology	6
6.1	AES-GCM	6
6.2	Encryption Process	6
6.3	Decryption Process	7
7	Threshold-Based Authorization	7
7.1	Approval Rules	7
7.2	Approval Workflow	7
8	Frontend Design	8
8.1	Features	8
8.2	Frontend Workflow	8
9	Working of the System	8
10	Applications of the Project	9
10.1	Banking Systems	9
10.2	Military Communication	9
10.3	Medical Systems	9
10.4	Enterprise Document Security	9
10.5	Cloud Secure Storage	9
11	Advantages and Limitations	10
11.1	Advantages	10

11.2 Limitations	10
12 Future Enhancements	10
13 Conclusion	11
14 References	11

1 Introduction

Secure communication has become one of the most important aspects of modern computing systems. Organizations, enterprises, and government systems continuously exchange confidential information through networks, making security a critical requirement. Traditional communication systems often rely only on user authentication and basic encryption, which may not provide adequate protection against unauthorized access, tampering, or insider attacks.

The objective of this project is to design and implement a secure file communication system using modern cryptographic techniques. The system provides encrypted file transfer combined with threshold-based authorization. Files remain encrypted until approvals are received from multiple authorized users.

The project uses AES-GCM encryption for secure storage and integrity verification. A multi-user approval mechanism is implemented to ensure that no single user can access sensitive information independently. The system is developed using React.js for frontend implementation and FastAPI for backend REST API development.

The project demonstrates concepts such as:

- Secure file transfer
- REST API communication
- Encryption and decryption
- Multi-user authorization
- Access control
- Integrity protection

The system provides a practical demonstration of secure communication workflows used in real-world enterprise applications.

2 Objectives

The major objectives of the project are:

- To implement secure encrypted communication using cryptographic techniques.
- To design a threshold-based authorization system.
- To ensure that sensitive files remain inaccessible until approvals are completed.

- To preserve the integrity and confidentiality of files during transmission.
- To implement secure backend communication using REST APIs.
- To provide a modern frontend interface for communication and approvals.
- To simulate a secure enterprise-style communication system.

The system also aims to demonstrate practical applications of cryptography in secure communication systems.

3 System Architecture

The project follows a client-server architecture. The system consists of two major modules:

3.1 Frontend Module

The frontend is implemented using React.js and provides the user interface for:

- Login functionality
- File upload
- Inbox viewing
- Approval management
- Secure file download

The frontend communicates with the backend through HTTP requests using Axios.

3.2 Backend Module

The backend is implemented using FastAPI. The backend is responsible for:

- Encryption and decryption
- Approval verification
- Access control
- File management
- REST API implementation

3.3 Architecture Flow

The communication flow is:

Sender → Encryption → Secure Storage → Approval Verification → Decryption →
Receiver

The backend acts as the central secure communication server.

4 Technologies Used

The project uses several modern technologies and libraries.

Technology	Purpose
React.js	Frontend user interface development
FastAPI	Backend REST API framework
Python	Backend programming language
Axios	Frontend-backend communication
AES-GCM	Encryption and integrity verification
Framer Motion	Frontend animations
React Dropzone	Drag-and-drop file upload
PyCryptodome	Cryptographic operations

These technologies provide both performance and security for the implementation.

5 REST API Design

The backend exposes REST APIs for communication between frontend and backend systems.

REST architecture is used because it is lightweight, scalable, and easy to integrate with modern frontend applications.

5.1 API Endpoints

Endpoint	Method	Purpose
/login	POST	User authentication
/send	POST	Upload encrypted file
/inbox	GET	Fetch receiver inbox

/pending	GET	Fetch pending approvals
/approve	POST	Approve encrypted message
/read	GET	Decrypt and download file

5.2 REST Communication Flow

The frontend sends HTTP requests to the FastAPI backend. The backend processes the request, performs cryptographic operations, and returns responses in JSON or file format.

This architecture allows separation of frontend and backend logic.

6 Encryption Methodology

The project uses AES-GCM encryption for securing files.

6.1 AES-GCM

AES stands for Advanced Encryption Standard. GCM (Galois Counter Mode) is a secure block cipher mode that provides:

- Confidentiality
- Integrity
- Authentication

AES-GCM is widely used in modern secure communication systems.

6.2 Encryption Process

The encryption workflow is:

1. Sender uploads a file.
2. Backend generates a random AES key.
3. File is encrypted using AES-GCM.
4. Nonce and authentication tag are generated.
5. Encrypted file is stored in secure memory.

6.3 Decryption Process

The decryption workflow is:

1. Receiver requests file access.
2. Approval status is verified.
3. AES-GCM verifies integrity using authentication tag.
4. Original file is decrypted.
5. Exact original file is returned to receiver.

7 Threshold-Based Authorization

The project implements threshold authorization using multi-user approvals.

7.1 Approval Rules

The following rules are implemented:

- Sender cannot approve their own file.
- Receiver cannot approve the file.
- Duplicate approvals are not allowed.
- Two different users are required for approval.

7.2 Approval Workflow

The approval process works as follows:

1. Sender uploads encrypted file.
2. Receiver sees encrypted message in inbox.
3. Two separate users approve the message.
4. System marks file as approved.
5. Receiver gains access to decrypted file.

This prevents unauthorized single-user access.

8 Frontend Design

The frontend provides an interactive and user-friendly interface.

8.1 Features

The frontend includes:

- Glassmorphism UI
- Drag-and-drop upload
- Dynamic inbox
- Approval dashboard
- Notification system
- Responsive design

8.2 Frontend Workflow

Users interact with the system through the dashboard. Actions such as sending files, approving requests, and downloading files are performed through API calls to the backend.

Animations and dynamic UI improve user experience.

9 Working of the System

The project workflow is:

1. User1 uploads a file for User2.
2. Backend encrypts the file using AES-GCM.
3. User2 sees the encrypted file in inbox.
4. User3 approves the message.
5. User4 approves the message.
6. System unlocks the file.
7. User2 downloads the original file.

The system preserves:

- Original file name
- Original file type
- Original file content

Example:

- PDF remains PDF
- PNG remains PNG
- DOCX remains DOCX

10 Applications of the Project

The project can be used in several real-world applications.

10.1 Banking Systems

Sensitive financial documents may require multi-level authorization before access.

10.2 Military Communication

Confidential mission files can be protected using threshold authorization.

10.3 Medical Systems

Patient records can be accessed only after approvals from authorized staff.

10.4 Enterprise Document Security

Corporate files can be securely shared using encrypted approval-based workflows.

10.5 Cloud Secure Storage

The project demonstrates concepts similar to secure cloud document vaults.

11 Advantages and Limitations

11.1 Advantages

- Strong encryption
- Secure communication
- Controlled decryption
- Multi-user authorization
- Integrity verification
- Modern frontend interface

11.2 Limitations

Current limitations include:

- No database persistence
- No JWT authentication
- No cloud deployment
- Uses in-memory storage

Data is lost when the backend server restarts.

12 Future Enhancements

Future improvements may include:

- RSA-based hybrid cryptography
- JWT authentication
- Database integration
- Cloud deployment
- WebSocket real-time updates
- Digital signatures

- Shamir Secret Sharing

These enhancements can convert the project into a production-level secure communication platform.

13 Conclusion

The Threshold-Based Secure File Communication System successfully demonstrates secure communication using modern cryptographic concepts.

The project integrates:

- AES-GCM encryption
- REST API communication
- Multi-user authorization
- Controlled decryption
- Secure frontend-backend interaction

The system provides practical exposure to secure communication design and demonstrates real-world applications of cryptography in enterprise systems.

The project also establishes a strong foundation for future improvements involving hybrid cryptography, digital signatures, and distributed secure communication systems.

14 References

1. FastAPI Documentation: <https://fastapi.tiangolo.com>
2. React.js Documentation: <https://react.dev>
3. PyCryptodome Documentation: <https://pycryptodome.readthedocs.io>
4. NIST AES Standard Documentation
5. OWASP Secure Communication Guidelines